

Extra Credit Homework Verilog/FPGA

olhartin@asu.edu

Objective

- In this exercise we will create a Neural Network trained on a FPGA
- We want it to be as realistic as possible, it has parameters that set the number of observations, features, output classes, and number of iterations
 - Remember these must be constants, they cannot be variables
- **Credit will be awarded based on the number of steps successfully completed**
- Let's start with compact notation, if total memory size of the dataset is a concern, then an entire row of ones is a waste

Some parameters and variables

- N_b is the number of bits in each number in X and W
- N_f is the number of features
- N_{obs} is the number of observations
- N_c is the number of classes

signed means it can represent positive and negative numbers, negative numbers are in 2's complement

- Y is the target
- Y_{pred} is the predicted value
- W is the weight
- b is the intercept, We can change the format of W to absorb the intercept b

Step 1: Modelf

- Modelf evaluates $y=XW+b$
- Then it quantizes y , a function is used, and that function can be changed to a different quantizer
- Write this code and test it with a testbench to make sure it works

```
module Modelf #(parameter Nb=8, Nf=3) (  
    input signed [Nb-1:0] X[0:Nf-1],  
    input signed [Nb-1:0] W[0:Nf-1],  
    input signed [Nb-1:0] b,  
    output logic signed [1:0] Ypred);  
  
    integer i;  
    logic signed [Nb-1:0] Y;  
    assign Zero={Nb{1'b0}}; ← logic signed [Nb-1:0] Zero;  
    //  
    function signed [1:0] quantization;  
        input signed [Nb-1:0] Y;  
    begin  
        if (Y>Zero)  
            quantization = 2'd1;  
        else  
            quantization = -2'd1;  
        end  
    endfunction  
    //  
    always_comb begin  
        Y = b; Ypred = 2'd0;  
        for(i=0;i<Nf;i++) begin  
            Y += X[i]*W[i];  
        end  
        Ypred = quantization(Y);  
    end  
endmodule
```

Step 1: Test bench

- This is a test bench
- Enter the code, run the simulation and capture an image of the simulation verification results

```
//  
// test Modelf  
//  
module test_Modelf();  
    localparam Nb=8, Nf=3;  
    logic signed [Nb-1:0] X[0:Nf-1];  
    logic signed [Nb-1:0] W[0:Nf-1];  
    logic signed [Nb-1:0] b;  
    logic signed [1:0] Y;  
    Modelf #(Nb, Nf) M1 ( X, W, b, Y);  
    initial begin  
        X = {-2, 4, -1};  
        W = {-14, -4, -2};  
        b = 2; #10;  
        $display("Y=%d", Y);  
    end  
endmodule
```

Step 2: This D Register and counter can be used to clock the modules to come

```
//  
// D register  
//  
module DReg #(parameter Nbits=8) (input clk, reset, enable, input [Nbits-1:0] D,  
                                   output logic [Nbits-1:0] Q);  
  
    always_ff @ (posedge clk or posedge reset)  
        if (reset)  
            Q <= {Nbits{1'b0}};  
        else if (enable)  
            Q <= D;  
  
endmodule  
  
//  
// counter, counts to MaxCnt-1 and generates clkout true for 1 clk cycle  
//  
module Counter #(parameter Nbc=3) (input clk, reset, enable, input [Nbc-1:0] MaxCnt,  
                                   output logic [Nbc-1:0] Count, output logic clkout);  
  
    logic rst, D;  
    DReg #(Nbc) D1 ( clk, reset|rst, enable, Count+{{Nbc-1{1'b0}},1'b1}, Count);  
    assign rst = (Count==MaxCnt) ? 1'b1 : 1'b0; // counts 0 1 2, ... MaxCnt-1  
    assign D = (Count==MaxCnt-1) ? 1'b1 : 1'b0;  
    DReg #(1) D2 ( clk, reset, 1'b1, D, clkout); // holds clkout for 1 clk cycle  
endmodule
```

Step 2: Test the counter

- Test the counter, nothing works if you haven't tested it
- Enter the code, run the simulation and capture an image of the simulation verification results

```
module test_counter();  
    localparam Nbc=4;  
    logic clk, reset;  
    logic [Nbc-1:0] MaxCnt, Count;  
    logic clkout;  
    Counter #(Nbc) C1 ( clk, reset, MaxCnt, Count, clkout);  
    initial begin  
        clk=1'b0; reset=1'b1; MaxCnt = 4'd10; #10;  
        reset = 1'b0;  
        repeat(15) begin  
            clk = 1'b0; #10;  
            clk = 1'b1; #10;  
            $display("Count %d clkout %b", Count, clkout);  
        end  
    end  
endmodule
```


Step 3: Go through all the observations

- Loop through observations on clock edges using the counter,
- The model will require $Nobs * Nf$ clock cycles
- This evaluate $Y = Xw + b$

```
module Modelo #(parameter Nb=8, Nobs=8'd8, Nf=3, Nbc=8) (  
    input clk, reset,  
    input signed [Nb-1:0] X[0:Nobs-1][0:Nf-1],  
    input signed [Nb-1:0] W[0:Nf-1], input signed [Nb-1:0] b,  
    output logic signed [1:0] Ypred[0:Nobs-1], output clkout);  
  
    localparam Zero={Nb{1'b0}};  
    integer i;  
    logic signed [1:0] Y_  
    logic [Nbc-1:0] Count;  
    // is Y driven for all Count at all times? need a ff?  
    Modelf #(Nb, Nf) M1 ( X[Count], W, b, Y_  
    Counter #(Nbc) C1 ( clk, reset, 1'b1, Nobs[Nbc-1:0], Count, clkout);  
    always_ff @ (posedge clk or posedge reset) begin  
        i = 0;  
        if (reset) begin  
            for(i=0;i<Nobs;i++)  
                Ypred[i] <= 2'd0;  
            end  
        else begin  
            Ypred[Count] <= Y_  
            // $display(" count %d Y_ %d Y %p ", Count, Y_, Y );  
        end  
    end  
endmodule
```


Step 3: Now test it

- Go through all observations evaluating each feature
- **Enter the code, run the simulation and capture an image of the simulation verification results**

```
module test_Modelo();
localparam Nb=8, Nobs=8'd7, Nf=3, Nbc=8;
logic signed [Nb-1:0] X[0:Nobs-1][0:Nf-1];
logic signed [Nb-1:0] W[0:Nf];
logic signed [Nb-1:0] b;
logic signed [1:0] Y[0:Nobs-1];
logic clk, reset, clkout;
int i;

Modelo #(Nb, Nobs, Nf, Nbc) M1 ( clk, reset, X, W[1:Nf], W[0], Y, clkout );
initial begin
    X = {{-2,4,-1},{4,1,-1},{1, 6, -1},{2, 4, -1},{6, 2, -1},{6, 1, -1},{1,2,-1}};
    W = {2, -14, -4, -2};
    clk = 1'b0; reset = 1'b1; #10;
    reset = 1'b0;
    for(i=0;i<=Nobs;i++) begin
        clk = 1'b0; #10;
        clk = 1'b1; #10;
        $display("Y=%p", Y);
    end
end
endmodule
```

Step 4: Now use the counter to go through all the classifications

```
module Modelc #(parameter Nb=8, Nobs=8, Nf=3, Nc=8'd3, Nbco=8, Nbcc=8) (  
    input clk, reset,  
    input signed [Nb-1:0] X[0:Nobs-1][0:Nf-1],  
    input signed [Nb-1:0] W[0:Nc-1][0:Nf],    // reverse Nc and Nf for reference  
    output logic signed [1:0] Ypred[0:Nc-1][0:Nobs-1], output clkoutc);  
  
    int i, j;  
    localparam Zero={Nb{1'b0}};  
    logic clkouto;  
    logic [Nbcc-1:0] Count;  
    logic signed [1:0] Y_[0:Nobs-1];  
    Modelo #(Nb, Nobs, Nf, Nbco) M2 ( clk, reset, X, W[Count][1:Nf], W[Count][0], Y_, clkouto );  
    Counter #(Nbcc) C1 ( clkouto, reset, 1'b1, Nc[Nbcc-1:0], Count, clkoutc);  
    always_ff @ (posedge clkouto or posedge reset) begin  
        i = 0; j = 0;  
        if (reset) begin  
            for(i=0;i<Nc;i++)  
                for(j=0;j<Nobs;j++)  
                    Ypred[i][j] <= 2'd0;  
        end  
        else begin  
            for(j=0;j<Nobs;j++)  
                Ypred[Count][j] <= Y_[j];  
        end  
    end  
end  
endmodule
```

- Count through all classes, evaluating now, evaluating each class, feature, and observation
- Changed W so that W_0 is b
- The rest of the weights are 1-Nf

Step 4: Now test this

- Because of the formulation of W , we don't need an intercept b

```
module test_Modelc();
localparam Nb=8, Nobs=8'd7, Nf=3, Nc=3, Nbco=8, Nbcc=8;
int i, j;
logic signed [Nb-1:0] X[0:Nobs-1][0:Nf-1];
logic signed [Nb-1:0] W[0:Nc-1][0:Nf];
logic signed [1:0] Y[0:Nc-1][0:Nobs-1];
logic clk, reset, clkoutc;
//
Modelc #( Nb, Nobs, Nf, Nc, Nbco, Nbcc) Mc ( clk, reset, X, W, Y, clkoutc);
initial begin
    X = {{-2,4,-1},{4,1,-1},{1, 6, -1},{2, 4, -1},{6, 2, -1},{6, 1, -1},{1,2,-1}};
    W = {{2, -14, -4, -2},{2, 8, 9, -9},{2, 9, -15, 9}};
    clk = 1'b0; reset = 1'b1; #10;
    reset = 1'b0;
    for(i=0;i<Nc;i++) begin
        for(j=0;j<=Nobs;j++) begin
            clk = 1'b0; #10;
            clk = 1'b1; #10;
        end
    end
    $display("test modelC Y=%p", Y);
end
endmodule
```

- Go through all observations and classifications evaluating all features in each clock cycle
- **Enter the code, run the simulation and capture an image of the simulation verification results**


```

module Modelct #(parameter Nb=8, Nobs=8, Nf=3, Nc=8'd3, Nbco=8, Nbcc=8) (
    input clk, reset,
    input signed [Nb-1:0] X[0:Nobs-1][0:Nf-1],
    input signed [Nb-1:0] W[0:Nc-1][0:Nf],
    input signed [1:0] Y[0:Nc-1][0:Nobs-1],
    input [3:0] etashift,
    output logic signed [1:0] Ypred[0:Nc-1][0:Nobs-1],
    output logic signed [Nb-1:0] dW[0:Nc-1][0:Nf],
    output clkoutc,
    output logic [Nb-1:0] TotalError );

    int i, j, k;
    int i_, j_, k_;
    localparam Zero={Nb{1'b0}}, One = {{Nb-1{1'b0}},1'b1};
    logic signed [1:0] zero2s;
    logic clkoutc;
    logic [Nbcc-1:0] Count;
    logic signed [1:0] Y_[0:Nobs-1];
    logic signed [1:0] Error[0:Nc-1][0:Nobs-1];

    assign zero2s = 2'd0;

    //
    // function to Take absolute value
    //
    function [Nb-1:0] abs;
        input [Nb-1:0] a;
    begin
        abs = a;
        if (a[Nb-1] == 1'b1)
            abs = ~a + {{Nb-1{1'b0}},1'b1}; // 2's compliment
    end
endfunction

    //
    // multiply 2, 1 bit signed numbers
    // (requires 2 bits)
    //
    function signed [1:0] onebitmult;
        input signed [1:0] a;
        input signed [1:0] b;
    begin
        onebitmult = 2'd1;
        if ((a>0 && b>0) || (a<0 && b<0))
            onebitmult = 2'd1;
        else if ((a<0 && b>0) || (a>0 && b<0))
            onebitmult = -2'd1;
    end
endfunction

```

Step 5:

- Now modify this module
- Create a function that takes the absolute value
- And a function that multiplies only 1 and -1
- Saving resources
- Stored as a 2 bit signed number

Step 5: Continued

- Go through all the classes but this time we are going to capture more

```
Modelo #(Nb, Nobs, Nf, Nbco) M2 ( clk, reset, X, W[Count][1:Nf], W[Count][0], Y_, clkouto );  
// count through all the classes  
Counter #(Nbcc) C1 ( clkouto, reset, 1'b1, Nc[Nbcc-1:0], Count, clkoutc);  
  
always_ff @ (posedge clkouto or posedge reset) begin  
    i = 0; j = 0;  
    if (reset) begin  
        for(i=0;i<Nc;i++)  
            for(j=0;j<Nobs;j++)  
                // reset to all 1 which shows plenty of error  
                Ypred[i][j] <= 2'd1;  
    end  
    else begin  
        for(j=0;j<Nobs;j++)  
            Ypred[Count][j] <= Y_[j];  
    end  
end
```

Step 5: Combinatorial block

- Initialization of all variables assigned in the always is necessary to prevent inferred latches, a pernicious type of synthesis error



```
always_comb begin
    // initialization
    TotalError = {Nb{1'b0}};
    for(i_=0;i_<Nc;i_++) begin
        for(j_=0;j_<Nobs;j_++)
            Error[i_][j_] = 2'd0;
        for(k_=0;k_<=Nf;k_++)
            dW[i_][k_] = Zero;
    end
end
```


Step 5: Continued

- This finds the total error in each prediction and the total error in the current model (W)
- Then it determines the gradient dW which is a matrix $N_f \times N_c$
- Some of the displays are cut off, they are for debugging and you can write your own or just leave them out, assuming you don't have to debug

```

for(i_=0;i_<Nc;i_++) begin
    for(j_=0;j_<Nobs;j_++) begin
        Error[i_][j_] = onebitmult(Y[i_][j_],Ypred[i_][j_]);
        if (Error[i_][j_]<zero2s)
            TotalError += One; // capture total error adding ca
    end
    for(k_=0;k_<=Nf;k_++) begin
        dW[i_][k_] = Zero;
        for(j_=0;j_<Nobs;j_++) begin
            if (k_==0) begin
                if (Error[i_][j_]<zero2s) begin // only sum for
                    if (Y[i_][j_]>zero2s)
                        dW[i_][k_] += One;
                    else
                        dW[i_][k_] -= One;
                end
                $display(" i_ %d k_ %d dW[i_][k_] %d Y %d Ypred %d", i_, k_, dW[i_][k_], Y[i_][j_], Ypred[i_][j_]);
            end
            else begin
                $display(" i_ %d j_ %d Error[i_][j_] %d Y %d Ypred %d", i_, j_, Error[i_][j_], Y[i_][j_], Ypred[i_][j_]);
                if (Error[i_][j_]<zero2s) begin // only sum for
                    if (Y[i_][j_]>zero2s)
                        dW[i_][k_] += X[j_][k_-1]>>>etashift;
                    else
                        dW[i_][k_] -= X[j_][k_-1]>>>etashift;
                end
                $display(" i_ %d k_ %d dW[i_][k_] %d Y %d Ypred %d", i_, k_, dW[i_][k_], Y[i_][j_], Ypred[i_][j_]);
            end
        end
    end
end
end
end
end
$display(" Y %p \n Ypred %p \n Error %p \n dW %p \n TotalError %d", Y, Ypred, Error, dW, TotalError);
endmodule

```

Step 5: Test Modelct

- Modelct evaluates the model and returns the gradient and total error
- Let's test it
- Enter the code, run the simulation and capture an image of the simulation verification results

```
module test_Modelct();
localparam Nb=8, Nobs=8'd7, Nf=3, Nc=3, Nbco=8, Nbcc=8;
int i, j;
logic signed [Nb-1:0] X[0:Nobs-1][0:Nf-1];
logic signed [Nb-1:0] W[0:Nc-1][0:Nf];
logic signed [1:0] Y[0:Nc-1][0:Nobs-1];
logic signed [1:0] Ypred[0:Nc-1][0:Nobs-1];
logic signed [Nb-1:0] dW[0:Nc-1][0:Nf];
logic clk, reset, clkoutc;
logic [3:0] etashift;
logic [Nb-1:0] TotalError;
//
Modelct #( Nb, Nobs, Nf, Nc, Nbco, Nbcc) Mct ( clk, reset, X, W, Y, etashift,
                                              Ypred, dW, clkoutc, TotalError );
// Modelc #( Nb, Nobs, Nf, Nc, Nbco, Nbcc) Mc ( clk, reset, X, W, Y, clkoutc);
initial begin
    etashift = 4'd1;
    X = {{-2,4,-1},{4,1,-1},{1, 6, -1},{2, 4, -1},{6, 2, -1},{6, 1, -1},{1,2,-1}};
    W = {{2, -14, -4, -2},{2, 8, 9, -9},{2, 9, -15, 9}};
    Y = {{1, 1, -1, -1, -1, -1, 1},{-1, -1, 1, 1, -1, -1, -1},{-1, -1, -1, -1, 1, 1, -1}};
    clk = 1'b0; reset = 1'b1; #10;
    reset = 1'b0;
    for(i=0;i<Nc;i++) begin
        for(j=0;j<=Nobs;j++) begin
            clk = 1'b0; #10;
            clk = 1'b1; #10;
        end
    end
    $display("test modelCt Y %p \n Ypred %p", Y, Ypred);
end
endmodule
```

Step 6: Now we are ready for Training

- Set up training, with input X and target Y, to determine BestW, the best weights, and determine the TotalError
- This is a training module

```
module train #(parameter Nb=8, Nobs=8, Nf=3, Nc=8'd3, Nbco=8, Nbcc=8, Nepochs=15)
    ( input clk, reset, enable,
      input signed [Nb-1:0] X[0:Nobs-1][0:Nf-1], input signed [1:0] Y[0:Nc-1][0:Nobs-1],
      input [3:0] etashift,
      output logic signed [Nb-1:0] BestW[0:Nc-1][0:Nf],
      output logic [Nbcc-1:0] Count, output clkoutc, clkoutt,
      output logic [Nb-1:0] TotalError);
    localparam Zero={Nb{1'b0}}, One = {{Nb-1{1'b0}},1'b1};
    integer i, j;
    logic signed [1:0] Ypred[0:Nc-1][0:Nobs-1];
    logic signed [Nb-1:0] dW[0:Nc-1][0:Nf];
    logic signed [Nb-1:0] W[0:Nc-1][0:Nf];
    logic [Nb-1:0] BestTotalError;
```


Step 6: Continued

```
//  
Modelct #(Nb, Nobs, Nf, Nc, Nbco, Nbcc) Mc ( clk, reset, X, W, Y, etashift,  
      Ypred, dW, clkoutc, TotalError);  
// goes through iterations to train model  
Counter #(Nbcc) Cl ( clkoutc, reset, (TotalError!=Zero), Nepochs[Nbcc-1:0], Count, clkoutt);  
//
```

- Go through Nepoch iterations evaluate the model and return the gradient dW and the TotalError associated with X, W, Y

```
always_ff @ (posedge clkoutc or posedge reset) begin  
    i = 0; j = 0;  
    if (reset) begin  
        for(i=0;i<Nc;i++) begin  
            for(j=0;j<=Nf;j++) begin  
                W[i][j] <= One;  
            end  
        end  
        BestTotalError = {Nb{1'b1}}; // big number  
    end  
    else if (enable & clkoutc) begin
```

- Reset the weights to a neutral value, can't use 0 for this model so we set all weights to 1
- Keep track of the best TotalError
- But Start with a really big number
- Note that it stops counting when the TotalError is 0 by disabling the counter

Step 6: Evaluate Weights

- Augment the weight by dW on each iteration
- Hold onto the weights that give the lowest error, this is called the pocket method

```
//
//      pocket method
//
    if (TotalError<BestTotalError) begin
        BestTotalError = TotalError;
        for(i=0;i<Nc;i++) begin
            for(j=0;j<=Nf;j++) begin
                BestW[i][j] <= W[i][j];
            end
        end
    end
//      update the weight matrix
    for(i=0;i<Nc;i++) begin
        for(j=0;j<=Nf;j++) begin
            W[i][j] <= W[i][j] + dW[i][j];
        end
    end
end
endmodule
```


Step 6: Test the training module

- Set up a problem and test whether you can find the weights using the modules we have put together

```
module test_Train();  
  
    localparam Nb=8, Nobs=7, Nf=3, Nc=8'd3, Nbco=8, Nbcc=8, Nepochs=200;  
    localparam Zero={Nb{1'b0}}, One = {{Nb-1{1'b0}},1'b1};  
    logic [Nbcc-1:0] Epoch;  
    logic clk, reset, enable;  
    logic signed [Nb-1:0] X[0:Nobs-1][0:Nf-1];  
    logic signed [1:0] Y[0:Nc-1][0:Nobs-1];  
    logic [3:0] etashift;  
    logic signed [Nb-1:0] W[0:Nc-1][0:Nf];  
    logic [Nb-1:0] TotalError;  
    logic clkoutc, clkoutt;
```

Step 6: continue testing training module

```
train #(Nb, Nobs, Nf, Nc, Nbco, Nbcc, Nepochs) Mt ( clk, reset, enable, X, Y,
    etashift, W, Epoch, clkoutc, clkoutt, TotalError);

initial begin
    etashift = 4'd1; enable = 1'b1;
    X = {{-2,4,-1},{4,1,-1},{1, 6, -1},{2, 4, -1},{6, 2, -1},{6, 1, -1},{1,2,-1}};
    //    initial weights
    W = {{2, -14, -4, -2},{2, 8, 9, -9},{2, 9, -15, 9}};
    Y = {{1, 1, -1, -1, -1, -1, 1},{-1, -1, 1, 1, -1, -1, -1},{-1, -1, -1, -1, 1, 1, -1}};
    reset = 1'b1; #10;
    reset = 1'b0;
    Epoch = {Nbco{1'b0}};
    repeat ( Nepochs ) begin
        //        create clock edges for all features, obs, classes
        repeat ( Nf * Nobs * Nc ) begin
            clk = 1'b0; #10;
            clk = 1'b1; #10;
        end
        $display("Epoch %d clkoutc %b clkoutt %b \n W %p ", Epoch, clkoutc, clkoutt, W);
        Epoch++;
        $display(" TotalError %d ", TotalError );
        if (TotalError==Zero) break;
    end
end
endmodule
```

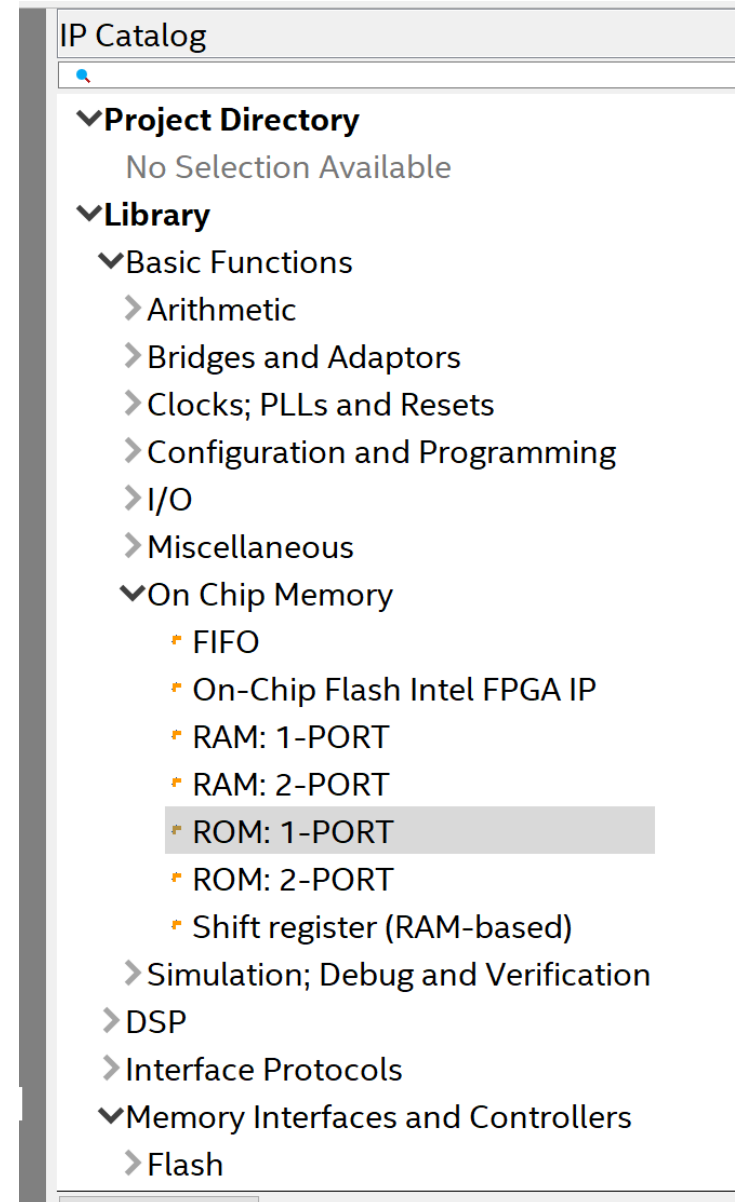
- Create clocks for features, observations, and classes
- Evaluate this for each iteration
- Enter the code, run the simulation and capture an image of the simulation verification results

Step 7: Next step, program it on an FPGA

- To put this simplest of models on an FPGA
- This presents specific problems
 - We need a way to load information into the FPGA
 - We need to place the resulting model on the FPGA
 - Test benches are for Simulation Verification, they won't synthesize on an FPGA
 - We will need to create something we can use directly on the FPGA
 - So, some background will be necessary

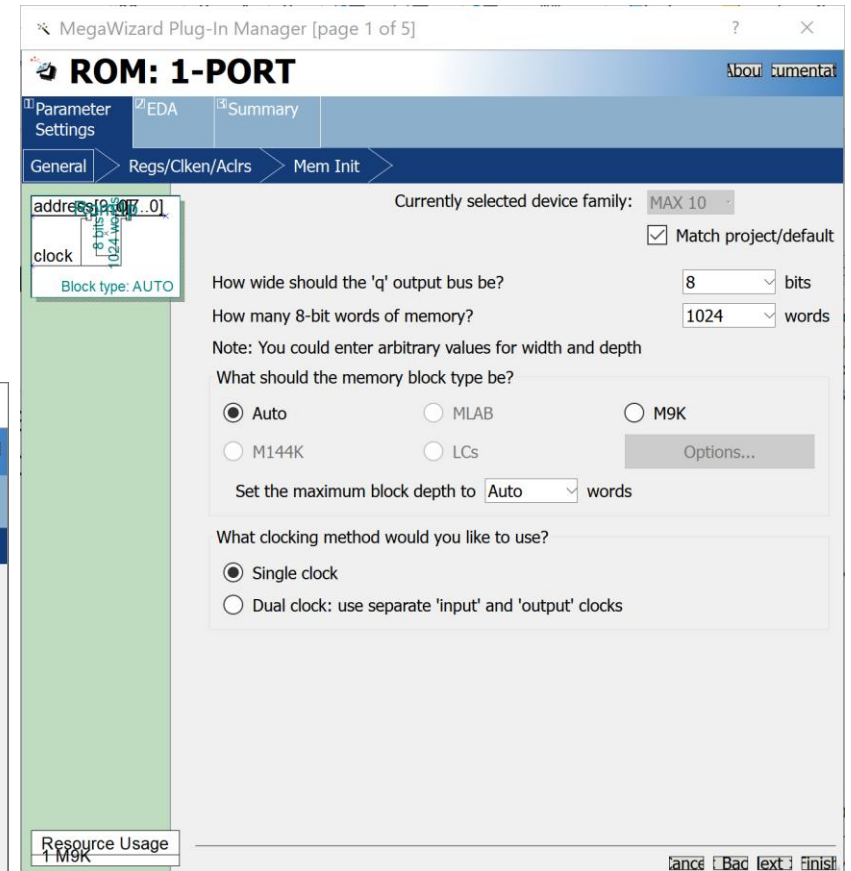
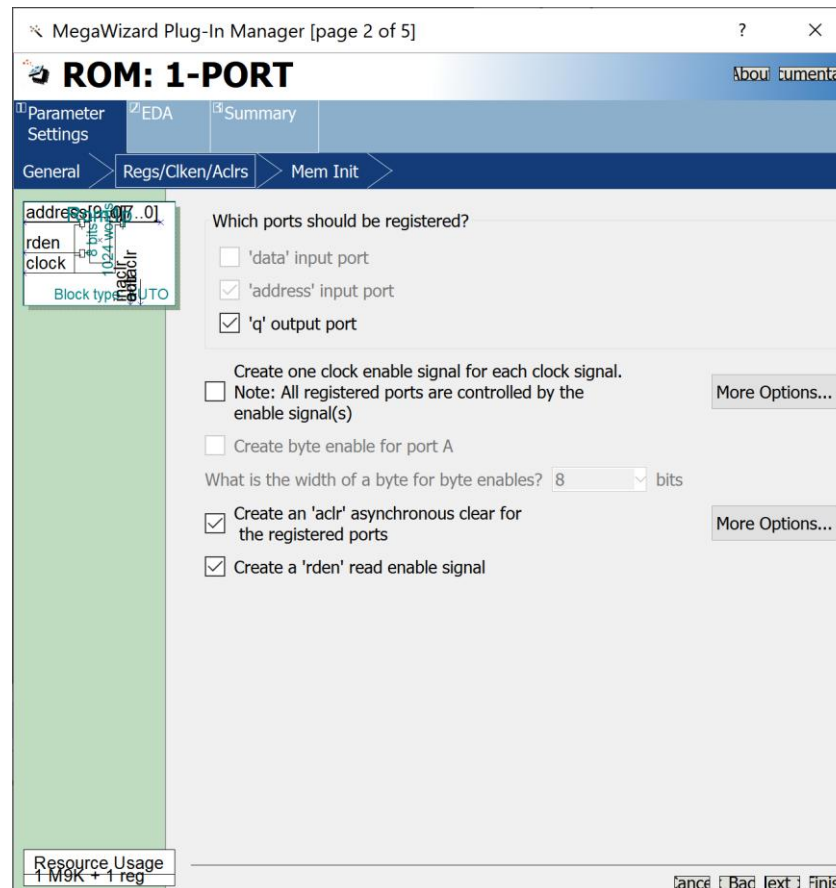
Use IP Catalog to add memory

- Create a Quartus project and in the IP catalog
 - Here we will use MAX10 FPGA with De10-Lite prototype board for device settings in this example, but you could change that
- Under IP-Catalog choose a 1 port ROM
 - In the pop up to save, give it a name – Rom1p, and select Verilog for the type of file



ROM 1-Port

- Select the number of bits and number of words
- Then hit next
- In the next screen you can create a clear and read enable



Initializing the ROM

- To initialize the ROM to some initial memory we can use a .mif (memory initialization file)
- “An ASCII text file (with the extension `.mif`) that specifies the initial content of a memory block (RAM or ROM), that is, the initial values for each address. This file is used during project compilation and/or simulation. You can create a Memory Initialization File in the Memory Editor, the In-System Memory Content Editor, or the Quartus[®] Prime Text Editor.”
- “A Memory Initialization File serves as an input file for memory initialization in the Compiler and Simulator. A Memory Initialization File contains the initial values for each address in the memory.”
- This is the format of the file

%demonstration of mif file%

DEPTH = 1024; -- The size of memory in words

WIDTH = 8; -- The size of data in bits

ADDRESS_RADIX = HEX; -- The radix for address

DATA_RADIX = BIN; -- The radix for data

CONTENT -- start of (address : data pairs)

BEGIN

000 : 11010111; -- memory address : data 215

001 : 11000001; -- 193

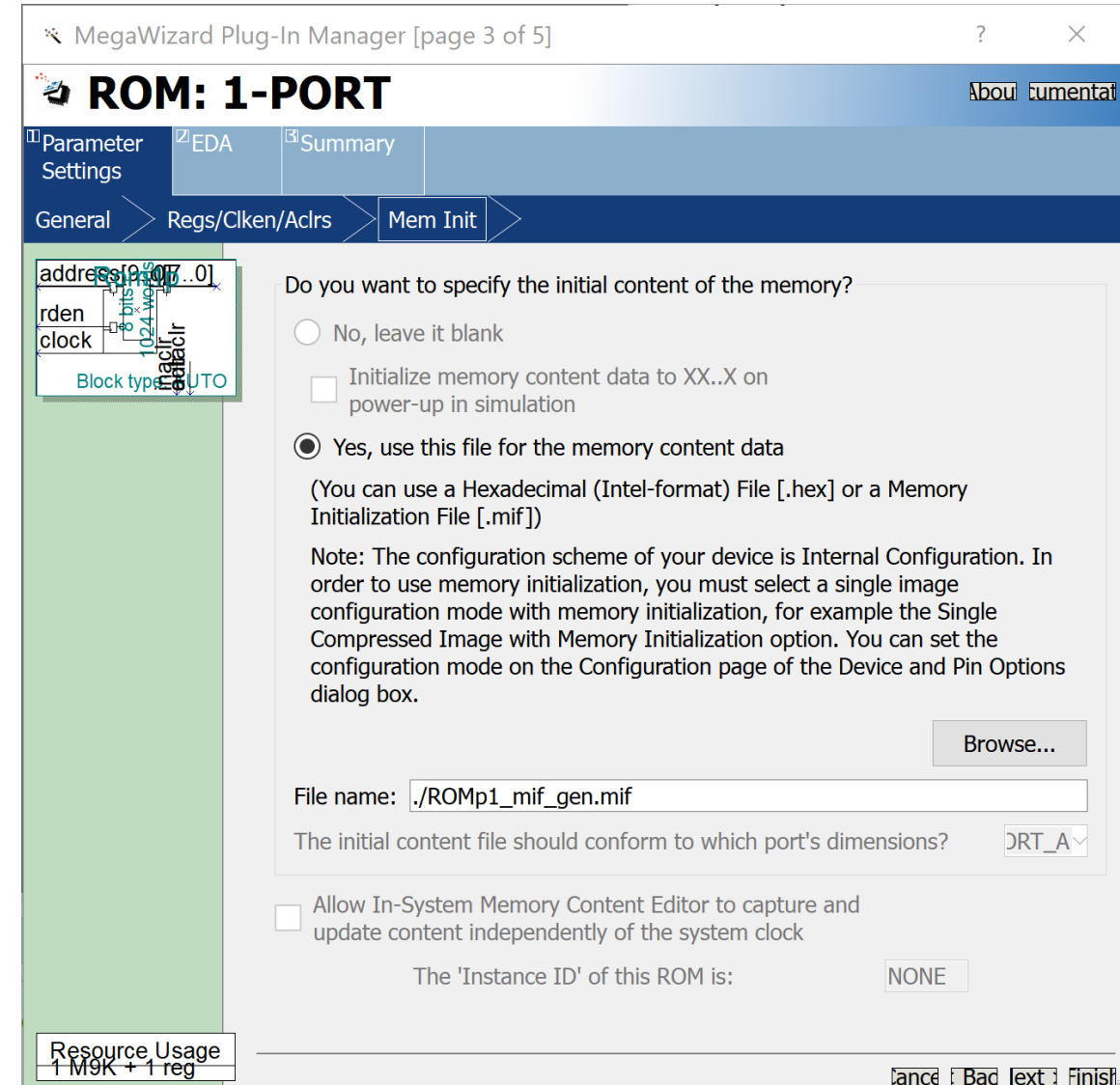
002 : 10100110;

...

END;

Initializing the ROM

- This file can be created in a number of ways including an excel spread sheet, export and change the file type from txt to mif
 - Place it in the project directory
- Browse to include that file, then next ... next and finish
- When prompted add the IP file to the project
- [Memory Initialization File \(.mif\) Definition \(intel.com\)](http://intel.com)



Compile in Quartus

- Ordinarily there would be some top level entity (module) that would use this memory which would be included in the project, but in this case we could just check our work by compiling Rom1p
- To compile this in Quartus set the top level entity to Rom1p
 - This is under assignments in the toolbar
- Also, you have to tell it to allow this type of initialization
 - Browse to Configuration in the path below, and choose the entry “Configuration Mode: Single uncompressed image with Memory Initialization” from the drop down
- Then you can compile

Assignments -> Device -> Device and Pin Options -> Configuration -> Configuration Mode: Single uncompressed image with Memory Initialization

Hit ok and then in the next screen hit ok

Options

×

Category:

General

EDA Tool Options

Fonts

Headers & Footers Settings

Internet Connectivity

Notifications

Libraries

IP Settings

IP Catalog Search Locations

Design Templates

License Setup

Preferred Text Editor

Processing

Tooltip Settings

Messages

Colors

Fonts

EDA Tool Options

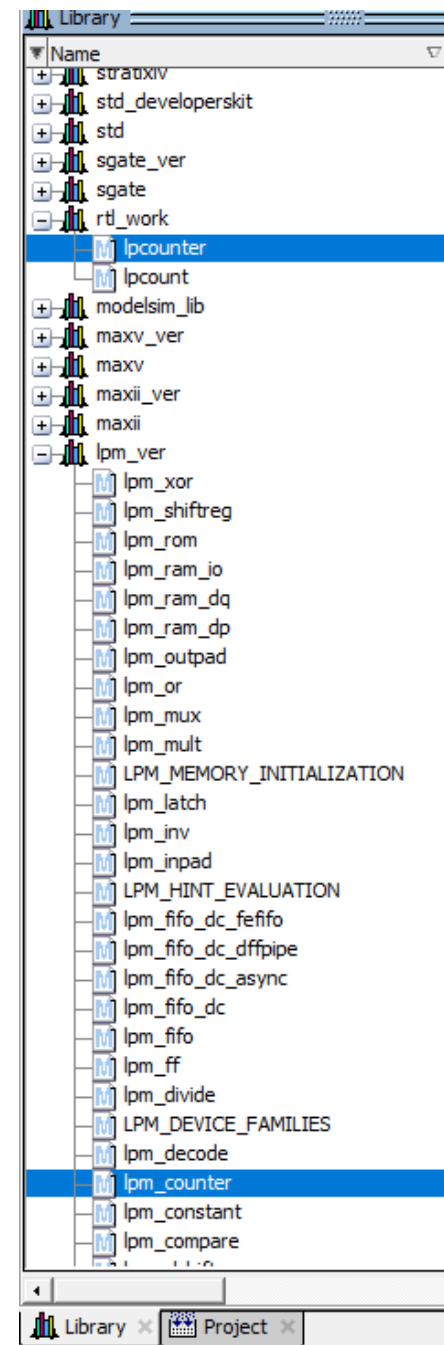
Specify the directory that contains the tool executable for each third-party EDA tool:

EDA Tool	Directory Containing Tool Executable
Precision Synthesis	...
Synplify	...
Synplify Pro	...
Active-HDL	...
Riviera-PRO	...
ModelSim	...
QuestaSim	...
ModelSim-Altera	C:\intelFPGA_lite\20.1\modelsim_ase\win32aloem...

OKCancelHelp

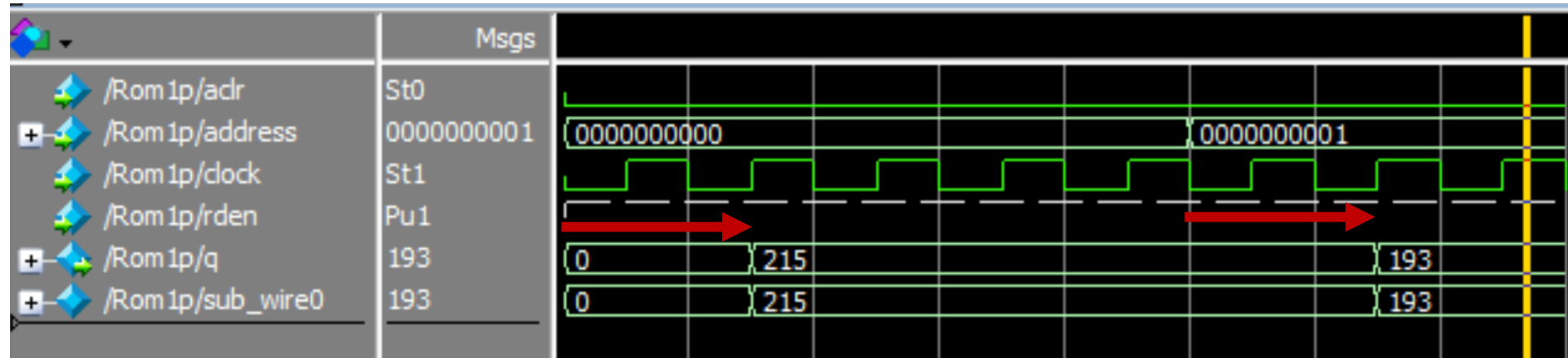
Simulation Verification

- Once again, you have to have Modelsim-Altera selected for EDA tool simulation as in section 1
- Under tools click Run Simulation Tool -> RTL Simulation
- A ModelSim Window will appear
- In the toolbar click Simulation->Start Simulation
- In window that pops up click library and then add
 - Browsing add altera_mf_ver
- Then in the window click design
 - Click work->Rom1p



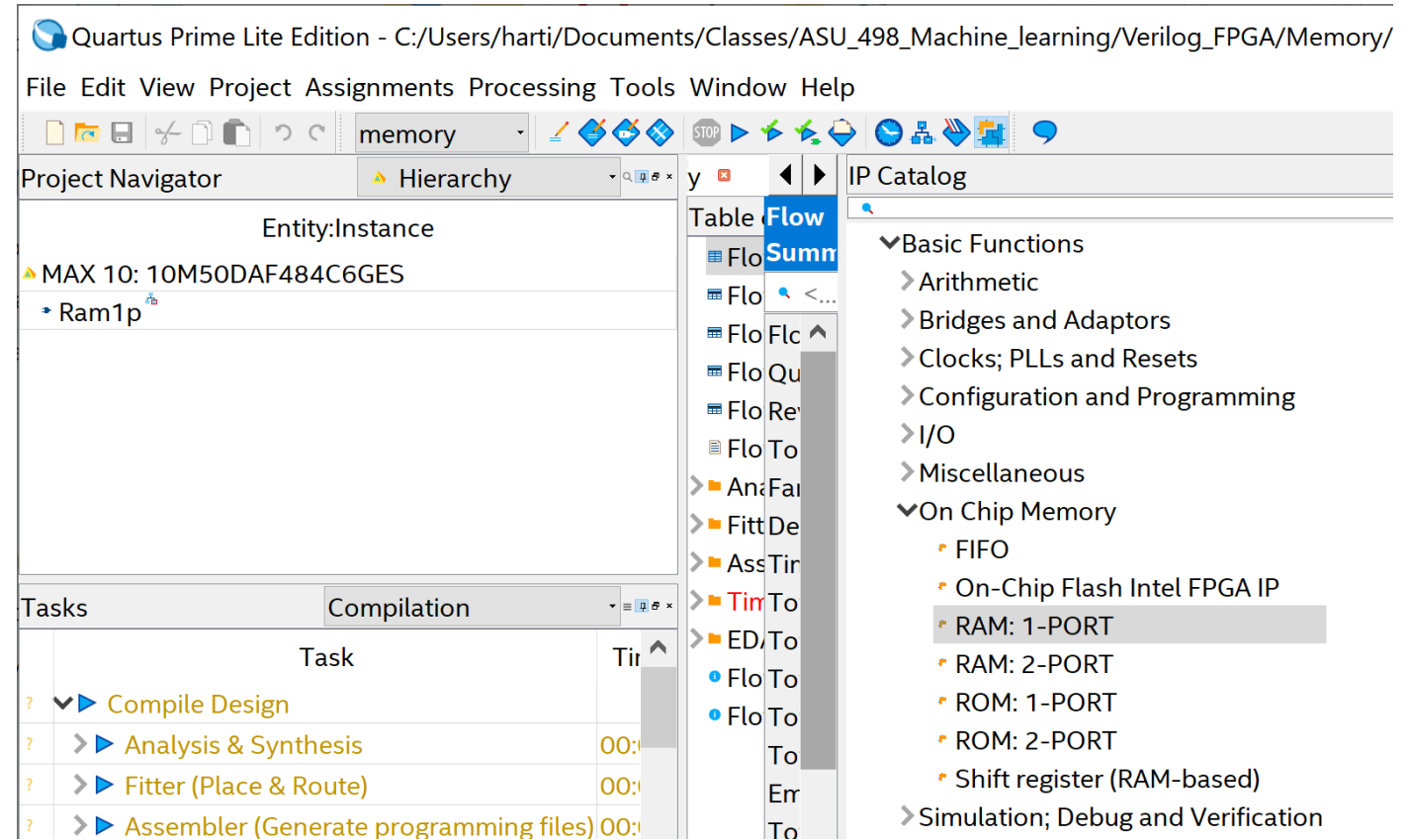
Simulation result

- Force the address to 0 and set the clock to a 100ps period clock
- Run the simulation for several cycles and then force the address to 1
- From the memory initialization file we know that address 0 is 215 and address 1 is 193
- That is what we see from the simulation
- We have successfully simulated pulling the value at address 0 and 1 from the ROM
- Note the one cycle delay (the value is retrieved on the second positive clock edge)



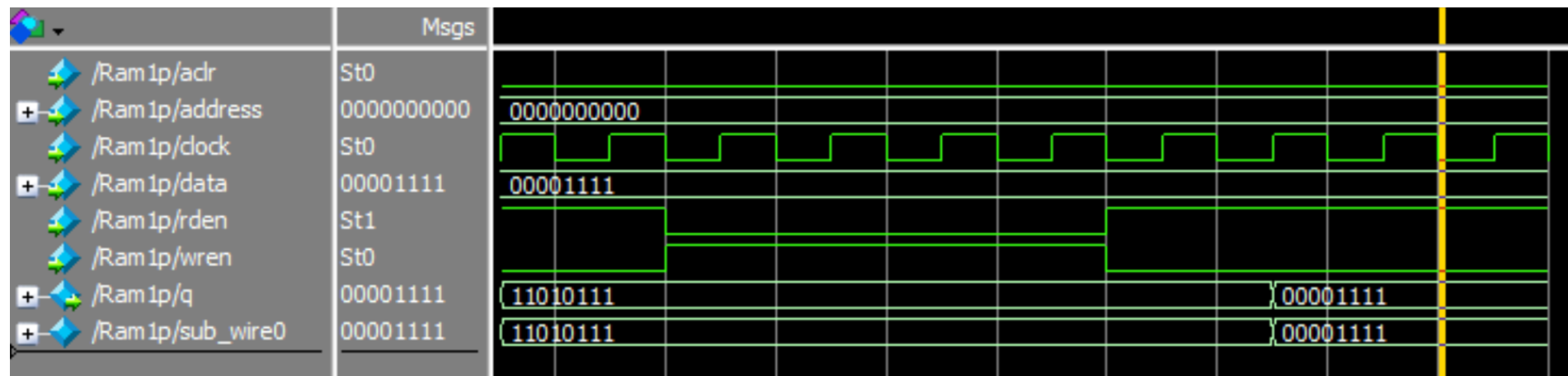
Now for RAM

- You can use a similar method to create a RAM
- Use module name Ram1p
- For simplicity using a 1 port RAM
- Will use the same memory initialization file but it isn't necessary, since ram can be written to and read from under execution
- Change the top-level-entity to Ram1p
- As before simulate



RAM test

- First read the initial value of address 0 by enabling read
- Then write the value 8'b0000_1111 to the RAM at address 0
- Then Read the value at that address verifying that it is now 8'b0000_1111
- This means we can use a memory initialization to initialize the memory in the RAM and then write new values and read them during execution
- Once again there is a full one cycle clock delay between enabling the read and observing the result in output q



Step 7: What does our synthesizable training module need to do

- We will use RAM
- Initialize the RAM with the data values X , we can do this by creating a .mif (memory initialization file)
- We need a finite state machine to do everything we need in order that they need to be done
- All finite state machines (FSM) need to be reset initially, which we can associate with a pushbutton or switch on a prototype board for our FPGA


```

module Train_driver #(parameter Nb=8, Nobs=10'd7, Nf=3, Nc=8'd3, Nbco=8, Nepochs=15)
    (input clk, reset, input [3:0] etashift, output logic clkoutc, clkoutt);
    localparam Nbcc = Nbco;
    localparam InitX = 4'd1, InitW = 4'd2, Train = 4'd3, Report = 4'd4, Store = 4'd5, Stop = 4'd6;
    localparam Zeroi = 10'd0, Onei = 10'd1, zero = 1'b0, one = 1'b1;
    localparam Zerod = {Nb{1'b0}}, Oned = {{Nb-1{1'b0}},1'b1};
    logic signed [Nb-1:0] X[0:Nobs-1][0:Nf-1];
    logic signed [1:0] Y[0:Nc-1][0:Nobs-1];
    logic signed [Nb-1:0] W[0:Nc-1][0:Nf];
    logic [Nb-1:0] TotalError;
    logic [Nbco-1:0] Epoch;
    logic Reset;
    logic clkoutc_;

    logic aclr, rden, wren, enable;
    logic clk_, clk_obs, clk_f;
    logic [7:0] data;
    logic [9:0] i, j, k, kXstart, kWstart, i_, j_;

    logic [3:0] NextState, State;

    assign kXstart = Zeroi; // first address of X
    assign kWstart = 10'd512; // first address of W

```

- kXstate is the location where X is stored in ROM

- kWstate is where it stores W in ROM

Step 7: Training code driver module

- clk is assigned to the internal FPGA clock

- reset is assigned to the a pushbutton on the prototype board

- etashift can be assigned to switches

- Clkoutc, and clkoutt can be assigned to LEDs

- This is done in pin assignment

Step 7: Training Driver

- Train trains the model, Modelc evaluated it,
- The counters counts the number of clock cycles to load X from RAM
- DReg holds the state

```
train #(Nb, Nobs, Nf, Nc, Nbco, Nbcc, Nepochs) Mt
    ( clk, Reset, enable, X, Y, etashift, W, Epoch, clkoutc, clkoutt, TotalError);

Modelc #( Nb, Nobs, Nf, Nc, Nbco, Nbcc) Mc ( clk, reset, X, W, Y, clkoutc_);
//
//     count clock cycles to determine when X has been filled
//
Counter #(Nbcc) Cobs ( clk_, reset, Nobs[Nbco-1:0], i, clk_obs);
Counter #(Nbcc) Cf ( clk_obs, reset, Nf[Nbco-1:0], j, clk_f);
//
//     read X from RAM when ren is true, write W to RAM when wren is true
//
Ram1p Ram1 ( aclr, k, clk, W[i_][j_], rden, wren, X[i][j]); // X must be 8 bits Nb
DReg #(Nb) D1 ( clk, reset, NextState, State);
//
```

Step 7: Combinatorial block

- Combinatorial block that determines NextState in the FSM
- InitX fills X from the RAM which was initialized from the .mif file when the FPGA was programmed
- Next it initializes W

```
//  
// determine next operating state  
//  
always_comb begin  
    i_ = Zeroi; j_ = Zeroi; Reset = 1'b0;  
    NextState = State; clk_ = zero;  
    k = Zeroi;  
    aclr = zero; rden = zero; wren = zero; enable = zero;  
    unique case(State)  
        //  
        //      fill X from RAM  
        //  
        InitX : begin  
            k = j * Nobs + i + kXstart;  
            NextState = InitX;  
            enable = zero;  
            clk_ = clk;  
            aclr = zero; rden = one; wren = zero;    // r  
            if ( clk_f == one ) begin  
                NextState = InitW;  
            end  
        end  
    end  
end
```

Step 7: Continued

- InitW state, initialize the W vector by setting Reset to True
- Train state sets enable for the training algorithm to True
- We can put some thing in Report to deliver result information somewhere like to the display

```
//  
//  
//  
InitW : begin  
    clk_ = zero;  
    rden = zero; // all done stop reading X  
    enable = one; // start training  
    NextState = Train;  
    Reset = 1'b1;  
end  
  
//  
//  
//  
Train the model, when finished turn enable to  
  
Train : begin  
    Reset = 1'b0;  
    NextState = Train;  
    enable = one; // start training  
    if (clkoutt==one) begin  
        enable = zero; // finished  
        NextState = Report;  
    end  
end  
  
//  
//  
//  
Report on the training accuracy  
  
Report : begin  
    Reset = 1'b0;  
    NextState = Store;  
end
```


Step 7: Store the model, W

- During the last state store the weights, which is the model, W back in the RAM starting at kWstate
- Enter the code give
- Program the FPGA in Quartus,
 - Capture an image of the flow summary
 - Capture an image showing it compiled in Quartus

```
//  
//  
//  
//  
// Store the trained weights and performance  
// back on RAM at kWstart  
  
Store : begin  
    Reset = 1'b0;  
    NextState = Stop;  
    wren = one;  
    rden = zero;  
    k = kWstart;  
    for (i_=Zeroi;i_<Nc;i_=i_+Onei) begin  
        for (j_=Zeroi;j_<Nf;j_=j_+Onei) begin  
            k += Onei;  
        end  
    end  
end  
endcase  
end  
endmodule
```